Java Multithreading & Concurrency - Complete Interview Notes

1. Core Fundamentals

What is Multithreading?

- **Multithreading** means multiple threads executing **within a single process**.
- Threads share the **same heap memory** and **process resources** (code, data, files, etc.).
- **Advantages**:
  - Better CPU utilization
  - Faster context switching than processes
  - Improved responsiveness and throughput
- **Use cases**:
  - Parallel processing
  - Asynchronous I/O (API calls, DB queries, file operations)
  - High-performance backend systems
  - Real-time applications

Process vs Thread

Feature	Process	Thread
Memory	Separate address space	Shared memory (same heap)
Communication	IPC (Pipes, Sockets, Shared Memory)	Direct shared memory (very fast)
Crash Impact	Only that process crashes	Can crash entire process
Creation Overhead	High (heavyweight)	Low (lightweight)
Context Switching	Expensive	Cheaper
Isolation	High	Low

Rule of thumb: Use **threads** when you need speed and shared data. Use **processes** when you need strong isolation.

Thread Lifecycle (States)

1. **New** → Thread object created but `start()` not called
2. **Runnable** → Ready to run (in thread scheduler queue)
3. **Running** → Currently executing on CPU
4. **Blocked / Waiting** → Waiting for lock, I/O, or notification
5. **Timed Waiting** → Waiting with timeout (`sleep()`, `wait(timeout)`)

6. **Terminated** → `run()` method completed or stopped

---

2. Thread Creation in Java

1. Extending `Thread` class (Not Recommended)

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running: " + Thread.currentThread().getName());
    }
}

// Usage
MyThread t = new MyThread();
t.start();
```

2. Implementing `Runnable` (Traditional Best Practice)

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread: " + Thread.currentThread().getName());
    }
}

// Usage
Thread t = new Thread(new MyRunnable());
t.start();
```


Modern way (Java 8+):

```
Thread t = new Thread(() -> {
    System.out.println("Lambda thread");
});
t.start();
```

3. Using `Callable<V>` (Returns result + can throw exception)

```
Callable<Integer> task = () -> {
    Thread.sleep(1000);
    return 42;
};

ExecutorService executor = Executors.newFixedThreadPool(2);
Future<Integer> future = executor.submit(task);

Integer result = future.get(); // blocks until done
```

Key Differences:

Feature	Runnable	Callable<V>
Return value	No	Yes (`V`)
Exception	Cannot throw	Can throw checked
Used with	Thread / Executor	ExecutorService

---

3. Synchronization & Race Conditions

Race Condition

When multiple threads access and modify shared data **concurrently** → inconsistent results.

`synchronized` Keyword

```
// 1. Synchronized Method
public synchronized void increment() {
    count++;
}

// 2. Synchronized Block (Preferred - smaller critical section)
public void increment() {
    synchronized(this) {   // or synchronized(obj)
        count++;
```

```
    }
}
```

- Every Java object has an **Intrinsic Lock** (Monitor).
- Only one thread can hold the lock at a time.

Important: `synchronized` is **reentrant** — same thread can acquire the lock multiple times.

---

4. Advanced Locks (`java.util.concurrent.locks`)

`ReentrantLock`

```
private final Lock lock = new ReentrantLock();

public void criticalSection() {
    lock.lock();
    try {
        // critical section
    } finally {
        lock.unlock();   // Always unlock in finally
    }
}
```


Advantages over `synchronized`:

- `tryLock()` — non-blocking attempt
- `tryLock(timeout)` — timed attempt
- `lockInterruptibly()` — can be interrupted
- Fairness option (`new ReentrantLock(true)`)
- Multiple condition variables possible

`ReadWriteLock`

```
ReadWriteLock rwLock = new ReentrantReadWriteLock();

Lock readLock = rwLock.readLock();
Lock writeLock = rwLock.writeLock();
```

- Multiple threads can read simultaneously.
- Only one writer at a time.
- Useful for caches, configuration, etc.

---

5. Thread Communication

`wait()`, `notify()`, `notifyAll()`

Rules:

- Must be called from **inside synchronized** block/method
- `wait()` releases the lock and goes to waiting state
- `notify()` wakes up **one** waiting thread
- `notifyAll()` wakes up **all** waiting threads

Classic Producer-Consumer:

```
synchronized(queue) {
    while (queue.isEmpty()) {
        queue.wait();           // Consumer waits
    }
    // consume
    queue.notifyAll();          // Notify producer
}
```

Better Alternative: Use `BlockingQueue` (e.g., `ArrayBlockingQueue`, `LinkedBlockingQueue`)

---

6. Executor Framework (Most Important for Interviews)

Never create threads manually in production code.

```
ExecutorService executor = Executors.newFixedThreadPool(10);

executor.submit(() -> {
    System.out.println("Task executed by: " + Thread.currentThread().getName());
});
```

```
executor.shutdown();            // Important!
```

Types of Executors

Executor Type	Use Case	Behavior
`newFixedThreadPool(n)`	Known number of threads	Fixed size
`newCachedThreadPool()`	Short-lived async tasks	Creates threads as needed, reuses
`newSingleThreadExecutor()`	Sequential tasks	Single thread
`newScheduledThreadPool()`	Periodic / delayed tasks	Scheduling support

`Future<V>`

```
Future<Integer> future = executor.submit(() -> compute());
Integer result = future.get();      // Blocking call
boolean done = future.isDone();
```

---

7. ThreadPoolExecutor Deep Dive

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    corePoolSize,      // 2
    maximumPoolSize,   // 5
    keepAliveTime,     // 60
    TimeUnit.SECONDS,
    new LinkedBlockingQueue<>(queueCapacity),  // 10
    new ThreadPoolExecutor.CallerRunsPolicy()  // Rejection policy
);
```

Execution Flow:

1. If threads < core → create new thread
2. If threads == core → put task in queue
3. If queue full and threads < max → create new thread
4. If max reached → apply **Rejection Policy**

Common Rejection Policies:

- `AbortPolicy` (default) → throws `RejectedExecutionException`
- `CallerRunsPolicy` → caller thread executes task
- `DiscardPolicy` → silently discard
- `DiscardOldestPolicy`

---

8. Java Memory Model (JMM)

Happens-Before Relationship

Guarantees **visibility** and **ordering** of actions between threads.

`volatile` Keyword

```
private volatile boolean flag = true;
```

Guarantees:

- **Visibility**: Changes made by one thread are immediately visible to others
- Prevents instruction reordering for that variable
- **Does NOT** provide atomicity for compound actions (e.g., `i++`)

Atomic Variables (Lock-Free)

```
AtomicInteger count = new AtomicInteger(0);

count.incrementAndGet();     // Atomic
count.compareAndSet(expected, update);  // CAS
```

CAS (Compare And Swap): Hardware-level atomic operation. Used by `Atomic*` classes.

---

9. Problems in Concurrency

1. Deadlock

Threads waiting for each other in a cycle.

Prevention:

- Always acquire locks in **same global order**
- Use `tryLock()` with timeout
- Avoid nested locks when possible

2. Livelock

Threads keep responding to each other but make no progress (e.g., polite retry with backoff).

3. Starvation

A thread is perpetually denied CPU/resources (usually due to priority or greedy threads).

---

10. Concurrent Collections

Collection	Use Case	Thread-Safe Feature
`ConcurrentHashMap`	High concurrency reads/writes	Lock striping
`CopyOnWriteArrayList`	Read-heavy, infrequent writes	Copy on write
`BlockingQueue`	Producer-Consumer	Blocking operations
`ConcurrentLinkedQueue`	Non-blocking queue	Lock-free

Why `ConcurrentHashMap` is better than `synchronized HashMap`?

- Better scalability (no single lock)
- Multiple threads can write to different segments
- `putIfAbsent()`, `computeIfAbsent()` etc. are atomic

---

11. CompletableFuture (Modern Java - Java 8+)

```
CompletableFuture.supplyAsync(() -> fetchData())
    .thenApply(data -> process(data))
    .thenAccept(result -> System.out.println(result))
    .exceptionally(ex -> {
        log.error(ex);
        return null;
    });
```

Key Features:

- Non-blocking asynchronous pipelines
- `thenApply`, `thenCompose`, `thenCombine`

- `allOf()`, `anyOf()`
- Better than `Future` for chaining

---

12. Must Practice Interview Problems

1. **Print numbers 1 to 10** using two threads (Even-Odd)
2. **Producer-Consumer** problem (with `wait/notify` and `BlockingQueue`)
3. **Thread-safe Singleton** (Double-Checked Locking + volatile)
4. Implement your own **Thread Pool**
5. **Rate Limiter** using Token Bucket / Leaky Bucket
6. Dining Philosophers (Deadlock example)
7. Implement a **Thread-safe Counter**

---

13. Real-World Usage in Backend Systems

- Parallel API calls (using `CompletableFuture`)
- Kafka / RabbitMQ consumers
- Async processing (emails, notifications, reports)
- Batch job processing
- Caching layers with `ReadWriteLock`
- Microservices internal task execution

---

14. Common Interview Questions

1. **Runnable vs Callable vs Future vs CompletableFuture**
2. `synchronized` **vs** `ReentrantLock`
3. `volatile` **vs** `AtomicInteger` **vs** `synchronized`
4. How does `ThreadPoolExecutor` work internally?
5. What is **Happens-Before** relationship?
6. Explain **CAS** and how `Atomic` classes use it.
7. Why is `ConcurrentHashMap` better than `Collections.synchronizedMap()`?
8. How to detect and prevent **Deadlock**?
9. Difference between **ExecutorService.submit()** and **execute()**?
10. What happens if you don't call `shutdown()` on ExecutorService?

---

15. Common Mistakes to Avoid

- Overusing `synchronized` (performance killer)
- Not shutting down `ExecutorService`
- Ignoring exception handling in threads

- Blocking calls inside async threads
- Sharing mutable state without proper synchronization
- Using `Thread.stop()`, `suspend()`, `resume()` (deprecated & dangerous)
- Forgetting `finally` block for lock release

Pro Tip for Interviews:

- Always mention **Executor Framework** first instead of manual threads.
- Prefer **higher-level constructs** (`CompletableFuture`, `BlockingQueue`, `ConcurrentHashMap`) over low-level `wait/notify`.
- Talk about **performance**, **scalability**, and **maintainability**.